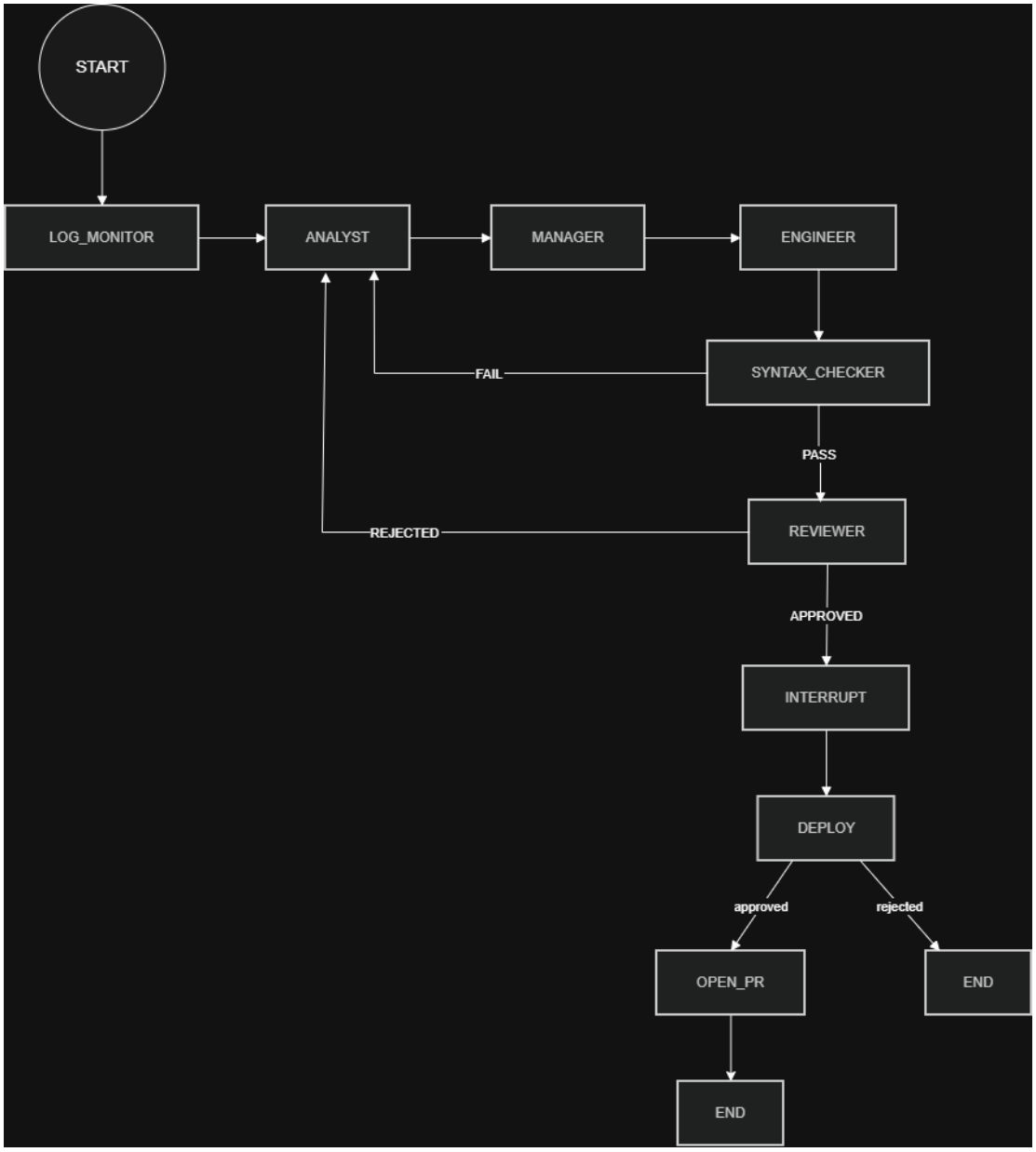
SentinalSRE – Engineering Notes (Local-First Inference)

SentinalSRE is a local-first multi-agent SRE system built on LangGraph with all reasoning performed through Ollama running Llama 3 locally. The goal was to automate incident triage while ensuring production logs never left the local machine.

1. Exact bottleneck where local inference broke down

The primary bottleneck wasn't raw model throughput it was context growth across multiple agents.

The pipeline performs several reasoning stages:



Initially, I considered forwarding each agent's complete output into the next stage. During testing this quickly became problematic because every node generated additional reasoning, causing prompt size to grow with each hop.

On a locally hosted Llama 3 model this resulted in:

- noticeably higher inference latency
- redundant reasoning being passed between agents
- unnecessary token usage
- lower responsiveness once multiple verification cycles occurred

The issue became more visible when the graph entered the retry loop after a failed syntax or semantic review, since every iteration increased prompt size further.

Rather than treating the graph as one long conversation, I redesigned it so each agent receives only the information it actually requires.

---

2. Trade-off made to keep latency acceptable

To keep inference practical on a local model, I deliberately traded context richness for bounded execution.

Instead of forwarding raw logs through the entire pipeline:

- production logs are ingested once
- sensitive data is removed through deterministic regex-based PII redaction
- the logs are compressed into a short sanitized summary
- downstream agents reason over that summary instead of the original log stream

This reduced prompt growth substantially while preserving the information necessary for root-cause analysis.

I also limited graph expansion in two additional ways:

- the reviewer/engineer correction cycle is capped at three iterations, preventing infinite reasoning loops
- syntax validation occurs before semantic review so obvious failures are rejected without spending another expensive LLM inference

Those decisions reduce unnecessary local inference while maintaining solution quality.

Additional engineering decisions

A few other design choices were made specifically because the system targets local inference:

- A single shared ChatOllama instance is reused across all nodes instead of repeatedly creating model instances.
- Human approval is inserted before deployment using LangGraph checkpoints, preventing wasted computation after generating pull requests.
- Latency is measured across pipeline stages to identify expensive nodes.

- GitHub operations execute through MCP instead of routing tool calls back through the language model.

Future improvements

The current implementation controls context growth through summarization and bounded retry loops, but there are several improvements I would make next:

- explicit token accounting at each graph node
- KV-cache reuse where supported by the serving engine
- adaptive summarization based on remaining context window
- memory profiling of Ollama GPU/CPU usage during long-running agent loops
- retrieval-based log chunking instead of summarizing entire incident logs

The current version demonstrates practical context management for local inference, while these additions would make resource utilization measurable instead of heuristic-driven.